

Procedural Terrain from Constraints and Graphs

Rohan Sinha

Sachith Shetty

Tianning Gao

`sinharoh@usc.edu`

`sachithc@usc.edu`

`gaotiann@usc.edu`

1 Introduction

1.1 Project Overview

We present a constraint-driven procedural terrain generation system that combines noise-based synthesis with graph-based spatial reasoning. Terrain is generated as scalar fields using a node-based graph and partitioned into Voronoi regions that define higher-level structure. Biomes are assigned using a Wave Function Collapse (WFC)-inspired constraint solver with entropy-based selection, constraint propagation, and backtracking to enforce adjacency rules. The resulting fields are converted into a triangle mesh with per-vertex attributes and rendered in real time using OpenGL, with additional effects such as river carving and smooth biome blending.

1.2 Main Contributions

- Constraint-driven terrain generation combining noise and graph-based reasoning
- Voronoi-based regional abstraction for coherent biome assignment
- WFC-inspired solver with constraint propagation and backtracking
- End-to-end pipeline from scalar fields to real-time rendering
- River carving and smooth biome blending for visual continuity

2 Background

2.1 Existing Methods

Procedural terrain generation commonly relies on spatial partitioning, constraint-based generation, and interpolation to introduce structure and coherence.

Voronoi Partitioning. Voronoi diagrams partition a domain into regions based on distance to a set of seed points:

$$V_i = \{p \mid \|p - s_i\| \leq \|p - s_j\|, \forall j \neq i\}$$

This method is widely used to generate natural-looking regions such as terrains and biomes [9]. Practical implementations are well illustrated in Red Blob Games [5].

Wave Function Collapse (WFC). WFC models generation as a constraint satisfaction problem [1]. Each cell maintains a set of possible states, which are iteratively reduced using entropy-based selection and constraint propagation. Extensions to larger structured environments are explored in [6].

Constraint Solving. More generally, constraint satisfaction problems (CSPs) are solved using propagation and backtracking [7], ensuring that all constraints between variables are satisfied.

Interpolation and Blending. To avoid discontinuities, interpolation methods such as smoothstep are used:

$$s(t) = t^2(3 - 2t)$$

which provides smooth transitions between regions [4]. Distance-based blending between neighboring regions is also commonly used in terrain generation [5].

2.2 Resources and References

We draw from both academic and practical resources. Red Blob Games [5] provides an intuitive explanation of Voronoi-based terrain generation. The Wave Function Collapse repository [1] and related tutorials [8] explain constraint propagation and entropy-based selection. Additional background is provided in procedural generation notes such as [3].

2.3 Comparison to Existing Projects

TerraForge3D [2] is a procedural terrain generation tool based on parameterized noise and height maps. Although it supports real-time terrain generation similar to our approach, it relies on parameter tuning rather than explicitly modeling terrain generation as a constraint-solving problem, highlighting the difference between parameter-driven and constraint-driven methods.

3 Methodology

3.1 Overview

Our terrain generation pipeline converts procedural noise into a structured, biome-annotated terrain through four main stages: (1) noise-based scalar field generation using a node-based graph, (2) terrain feature composition and river carving, (3) Voronoi-based spatial partitioning and biome assignment using a constraint solver, and (4) mesh generation and rendering. This design separates local detail generation from global structure, allowing explicit control over biome coherence.

3.2 Noise Synthesis and Scalar Fields

The terrain is generated as a set of scalar fields over a 2D grid. A user-defined node graph is compiled into a topologically sorted structure, allowing efficient single-pass evaluation. A Position node emits world-space (x, z) coordinates, which are optionally warped using FBM-based domain distortion. These coordinates are passed through noise generators (Simplex,

Perlin, FBM, ridged FBM) to produce continuous scalar fields in $[0, 1]$.

Each node output is cached per grid cell, avoiding recomputation. This produces base fields such as elevation, temperature, and moisture, which are used in later stages.

3.3 Terrain Feature Composition

Instead of generating terrain from a single noise function, the system uses specialized nodes for different landforms (Mountain, Valley, Plains, Plateau). Each node outputs both a height contribution and an influence weight. A Blend node combines these outputs to produce the final heightfield.

River networks are generated using a priority-flood drainage algorithm. High-accumulation cells are traced downstream to form river paths, which are then smoothed and carved into the terrain with a continuous falloff.

3.4 Voronoi Partitioning and Biome Assignment

To introduce large-scale structure, the terrain is partitioned into jittered Voronoi regions. Each region stores its center, associated grid points, and adjacency relationships.

Biome assignment is formulated as a constraint satisfaction problem and solved using a WFC-inspired algorithm. Each region begins with all possible biomes. The solver iteratively selects the region with the fewest valid options (lowest entropy), assigns a biome using weighted sampling, and propagates constraints to neighboring regions. If a contradiction occurs (i.e., no valid biome remains), the solver backtracks to a previous state.

Weights are influenced by environmental heuristics (temperature and moisture) and neighboring biome types to encourage spatial continuity.

After solving, each grid point inherits the biome of its region. Smooth transitions are produced by interpolating between the two nearest regions using a smoothstep function.

3.5 Rendering and Mesh Generation

The heightfield is converted into a triangle mesh by connecting grid points into indexed triangles. Per-vertex attributes include position, normal, slope, and biome weights. Normals

Algorithm 1 Biome Assignment using WFC-inspired Solver

```
1: while there are uncollapsed Voronoi cells do
2:    $cell \leftarrow$  cell with fewest valid biome options
3:    $candidates \leftarrow$  possible biomes for  $cell$ 
4:   for each biome in  $candidates$  do
5:     compute weight from climate heuristics
6:     if neighbors share similar biome type then
7:       increase weight
8:   select biome using weighted random sampling
9:   assign biome to  $cell$ 
10:  propagate constraints to neighbors
11:  if any cell has no valid biome then
12:    revert to previous state (backtrack)
```

are computed from height gradients, and directional lighting is applied in the shader to enhance terrain features. The final rendering blends biome colors based on weights and slope, producing visually continuous terrain with clear elevation structure.

3.6 Implementation and Course Concepts

The system is implemented in C++17 with custom noise functions and a node-based procedural graph system. Rendering uses OpenGL 3.3 with SDL2. The project integrates four core course concepts: (1) **Triangles and Meshes**—the heightfield is triangulated into an indexed mesh with per-vertex normals, slope, and biome weights uploaded to GPU buffers (VAO/VBO); (2) **Texture Mapping**—grass, rock, sand, and snow materials are blended based on biome weights and slope, with smoothstep interpolation at Voronoi boundaries; (3) **Noise**—Simplex, Perlin, FBM, and ridged FBM are composed with domain warping; and (4) **Procedural Generation**—all content is procedural, from the node-graph heightfield to procedurally generated rivers based on drainage simulation and WFC-assigned biomes. AI-assisted tools, including OpenAI Codex (GPT-5.4) and Gemini 3.1 Pro, were used for limited tasks such as boilerplate code generation and debugging.

4 Evaluation

4.1 Results

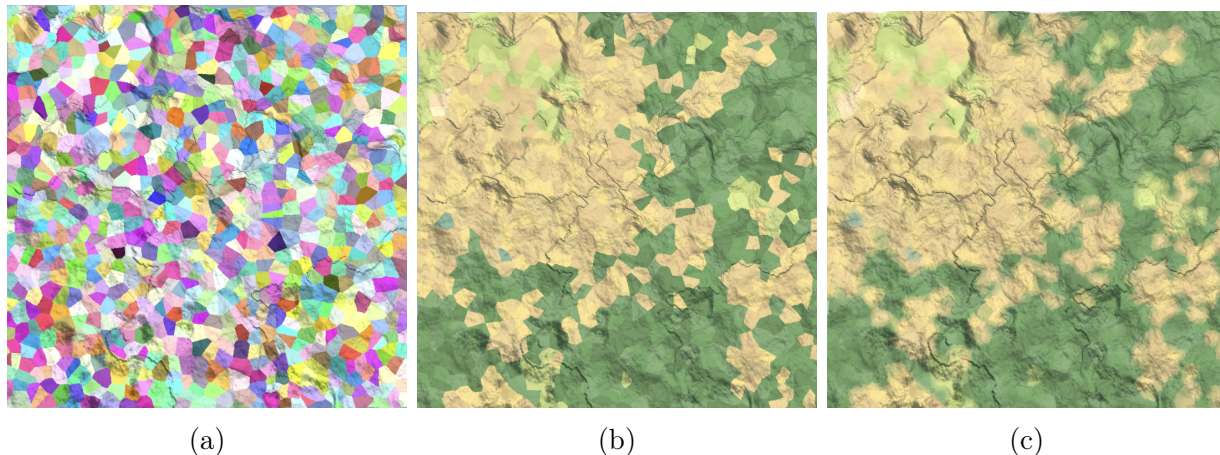


Figure 1: (a) Voronoi partition, (b) WFC assignment, (c) blended transitions.

Figure 1a shows the Voronoi partition used to divide the terrain into regions based on nearest jittered centers. Figure 1b shows biome assignment after applying the constraint solver, where adjacency rules produce coherent large-scale regions instead of random patterns. Figure 1c shows the effect of distance-based interpolation, which smooths sharp polygonal boundaries while preserving structure.

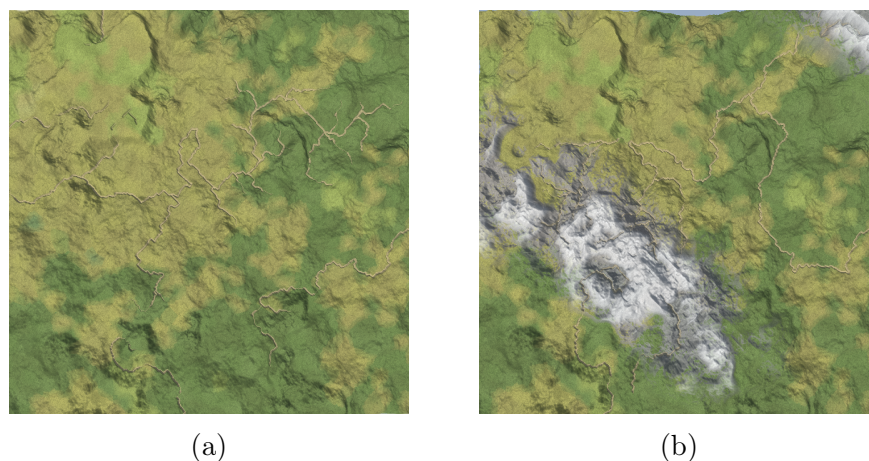


Figure 2: (a) Final render, (b) mountain variant.

Figure 2a shows the final rendered terrain. Compared to the blended image, it incorpo-

rates slope-based normals and lighting, making terrain features such as ridges and valleys more pronounced. Figure 2b shows a variant generated by adding a mountain node to the procedural graph, resulting in increased elevation variation.

4.2 Contribution of Each Component

The Voronoi partitioning groups the terrain into regions by assigning the nearest-center, eliminating noisy biome labels per-pixel and enabling regional-level decisions (Figure 1a). The WFC-inspired solver then assigns a biome to each region under adjacency constraints; propagation prunes incompatible options and backtracking resolves conflicts, resulting in contiguous and valid biome clusters (Figure 1b). Temperature and moisture fields bias the selection toward plausible climates.

Blending computes a weighted mix of the two nearest regions using smoothstep, removing hard Voronoi edges while preserving large-scale structure (Figure 1c). The final render differs from the blended image because it uses normals from height gradients and directional lighting, adding geometric shading and depth (Figure 2a). The node-based graph makes the system extensible: adding a mountain node increases elevation variation without modifying the underlying constraint solver or rendering pipeline (Figure 2b).

4.3 Strengths

The constraint-driven approach produces coherent large-scale biome regions by enforcing adjacency rules through propagation and backtracking, preventing incompatible transitions that commonly occur in noise-based methods. Additionally, the graph-based design makes the system highly extensible, allowing terrain features to be modified by adding or adjusting nodes, as demonstrated by the mountain variant.

4.4 Limitations

The Voronoi partition introduces visible large-scale region artifacts; although blending reduces boundary sharpness, the underlying region structure can still influence transitions. Additionally, the solver uses a simplified entropy measure (count of valid states) instead of

Shannon entropy and relies on backtracking, which may lead to suboptimal collapse ordering and increased computation time for larger terrains.

5 Conclusion

5.1 Key Takeaways

We found that mixing procedural noise with graph constraints gives both nice detail and overall structure. Using Voronoi regions made biome placement more stable and less sensitive to noise. Our adapted Wave Function Collapse helped keep neighboring biomes consistent. The node-based graph made it easy to combine different terrain features. Adding rivers and drainage made the terrain feel more realistic.

5.2 Team Contributions

- **Sachith Shetty:** Developed the node-based graph system, including the graph UI, compilation, and execution pipeline, and integrated it with terrain synthesis.
- **Rohan Sinha:** Voronoi region generation, WFC-inspired constraint solver (propagation, backtracking), blending and interpolation.
- **Tianning Gao:** Noise-based scalar fields, terrain feature nodes, river generation, mesh construction, OpenGL rendering pipeline.

5.3 Future Work

Looking ahead, several directions could extend this work. First, the biome constraint system could be expanded to support hierarchical biomes (e.g., sub-biomes within forests) and more complex adjacency rules that incorporate elevation, moisture, and temperature. Performance could be improved by parallelizing the WFC solver or offloading constraint propagation to the GPU, enabling larger terrain grids in real time.

References

- [1] Maxim Gumin. *Wave Function Collapse*. URL: <https://github.com/mxgmn/WaveFunctionCollapse>.
- [2] Jaysmito101. *TerraForge3D*. URL: <https://github.com/Jaysmito101/TerraForge3D>.
- [3] Rory Lyons. *Procedural Generation Notes*. URL: <https://axelftv.github.io/content/university/Rory-Lyons-Procedural-Generation.pdf>.
- [4] OpenGL. *Smoothstep Function*. URL: <https://registry.khronos.org/OpenGL-Refpages/gl4/html/smoothstep.xhtml>.
- [5] Amit Patel. *Terrain from Noise*. URL: <https://www.redblobgames.com/maps/terrain-from-noise/>.
- [6] “Procedural Generation of 3D Maps with Wave Function Collapse: Optimization and Advanced Constraints”. In: *Eurographics* (2021). URL: <https://diglib.eg.org/server/api/core/bitstreams/28237711-7df0-49ef-a6d2-b65aa163bb75/content>.
- [7] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. 2010.
- [8] Various. *Wave Function Collapse Tutorial*. URL: <https://robertheaton.com/2018/12/17/wavefunction-collapse-algorithm/>.
- [9] Steven Worley. “A cellular texture basis function”. In: *SIGGRAPH* (1996).